

TestPlatform

Rapport d'Execution de Campagne

Campagne #19

Projet: Projet #17

Version: N/A

Environnement: Environnement #12

Date: 03/06/2026 10:24

ECHEC

Resume executif

Taux de reussite



Evolution vs campagne precedente: N/A

Premiere campagne

Total	Succes	Echecs	Erreurs	Duree
1	0	1	0	210.0 s

Informations du contexte

Projet	Projet #17
Description	N/A

Suites

Nom	Type	Depot Git	Branche
Suite UNIT	UNIT	N/A	main

Cas de test

Titre	Type	Priorite	Risque	Script
Test #32	UNIT	N/A	N/A	IA

Environnement

URL Web	N/A
URL API	N/A
Repository	https://github.com/ilyeschaabani/Automatisation_intelligente_des_tests_applicatifs
Branche	main
Base de donnees	POSTGRESQL

Resultats details

Test #32

FAILURE

Type	UNIT
------	------

Duree	8.8 s
-------	-------

Script	IA
--------	----

Erreur	Maven exit code : 1
--------	---------------------

Analyse IA

Test #32

Logs (extrait)	<pre>R] [ERROR] For more information about the errors and possible solutions, please read the following articles: [ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException</pre>
Analyse IA	La cause probable de l'échec du test est la présence de warnings dans le code source. Dans votre cas, il semble que les fichiers `CampaignService.java` génèrent des avertissements non résolus car ils utilisent des opérations non vérifiées ou non sûres. Cela peut avoir une incidence sur le succès du test, car si les tests échouent en raison de ces avertissements, l'erreur d'exécution est probablement due à des erreurs réelles dans votre code source et non dans le code de vos tests. Pour corriger le problème : 1. Modifier la configuration Maven pour ignorer les avertissements non résolus. Vous pouvez y parvenir en ajoutant la ligne suivante à votre fichier `pom.xml`: ``xml <properties> <maven.compiler.source>1.8</maven.compiler.source> <maven.compiler.target>1.8</maven.compiler.target> <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding> <skipTests>false</skipTests> </properties> ``2. Vous pouvez également ajouter une nouvelle configuration de compilation pour ignorer les avertissements non résolus dans le fichier `pom.xml`: ``xml <plugin> <groupId>org.apache.maven.plugins</groupId> <artifactId>maven-compiler-plugin</artifactId> <version>3.8.1</version> <!-- Vérifier la version du plugin --> <configuration> <source>\${java.version}</source> <target>\${java.version}</target> <showDeprecation>true</showDeprecation> <fork>true</fork> <compilerArgument>-Xlint:none</compilerArgument> <!-- cette ligne désactive tous les avertissements --> </configuration> </plugin> `` 3. Enfin, vérifiez que votre projet fonctionne correctement en exécutant `mvn clean install` dans la console ou dans votre IDE Maven correspondant. Si tout se passe bien, le build devrait aboutir sans avertissements et échecs de tests. Suggestion d'amélioration du test : si vous n'avez pas besoin des opérations non vérifiées ou non sûres pour exécuter les tests, il est possible de les supprimer complètement pour améliorer la lisibilité et la maintenabilité du code. Cependant, cela n'est généralement pas recommandé car le risque d'avoir des bugs non détectés dans votre logique métier reste important.

Evaluation UX

Aucun test UX execute pour cette campagne

Metriques

Top 5 des tests les plus lents

Test	Suite	Duree
Test #32	Suite UNIT	8.8 s

Temps d'execution par type

Type	Duree moyenne
UNIT	8.8 s

Tests instables

Aucun test instable detecte

Recommandations

- Test #32: La cause probable de l'échec du test est la présence de warnings dans le code source. Dans votre cas, il semble que les fichiers `CampaignService.java` génèrent des avertissements non résolus car ils ...

Annexe - Scripts generes par IA

Test #32

```
package suites.unit;

import com.pfe.platform.ms_gestion.dto.request.CreateTestCaseRequest;
import com.pfe.platform.ms_gestion.dto.response.TestCaseResponse;
import com.pfe.platform.ms_gestion.entity.Project;
import com.pfe.platform.ms_gestion.entity.ProjectMember;
import com.pfe.platform.ms_gestion.entity.TestCase;
import com.pfe.platform.ms_gestion.entity.TestSuite;
import com.pfe.platform.ms_gestion.repository.ProjectMemberRepository;
import com.pfe.platform.ms_gestion.repository.TestCaseRepository;
import com.pfe.platform.ms_gestion.repository.TestSuiteRepository;
import com.pfe.platform.ms_gestion.security.SecurityUtils;
import com.pfe.platform.ms_gestion.service.LlmService;
import com.pfe.platform.ms_gestion.service.SkeletonExtractorService;
import com.pfe.platform.ms_gestion.service.TestCaseService;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockedStatic;
import org.mockito.Mockito;
import org.mockito.MockitoAnnotations;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;
import java.util.Optional;

public class TestCaseServiceTest {

    @Mock private TestCaseRepository testCaseRepository;
    @Mock private TestSuiteRepository testSuiteRepository;
    @Mock private ProjectMemberRepository projectMemberRepository;
    @Mock private LlmService llmService;
    @Mock private SkeletonExtractorService skeletonExtractorService;
    @InjectMocks private TestCaseService testCaseService;

    private AutoCloseable mocks;
    private MockedStatic<SecurityUtils> mockedSecurity;

    @BeforeMethod
    public void setUp() {
        mocks = MockitoAnnotations.openMocks(this);
        mockedSecurity = Mockito.mockStatic(SecurityUtils.class);
        mockedSecurity.when(SecurityUtils::getCurrentUserId).thenReturn(1L);
    }

    @AfterMethod
    public void tearDown() throws Exception {
        mockedSecurity.close();
        mocks.close();
    }

    @Test
    public void testAddTestCase() {
        // Arrange
        Long suiteId = 1L;

        Project project = new Project();
        project.setId(10L);
        project.setName("Test Project");

        ProjectMember member = new ProjectMember();
        member.setRole(ProjectMember.Role.ADMIN);
        member.setUserId(1L);
```

```
member.setProject(project);

TestSuite suite = new TestSuite();
suite.setId(suiteId);
suite.setType(TestSuite.TestType.UNIT);
suite.setProject(project);

CreateTestCaseRequest request = new CreateTestCaseRequest();
request.setTitle("Login test");
request.setType("UNIT");
request.setRiskLevel("LOW");
request.setPriority(1);
request.setDescriptionAI("Test the add method");

Mockito.when(testSuiteRepository.findById(suiteId))
    .thenReturn(Optional.of(suite));
Mockito.when(projectMemberRepository.findByIdAndUserId(
Mockito.any(), Mockito.eq(1L)))
    .thenReturn(Optional.of(member));
Mockito.when(llmService.generateTestCode(
Mockito.any(), Mockito.any(), Mockito.any(), Mockito.any()))
    .thenReturn("public class FakeTest {}");
Mockito.when(testCaseRepository.save(Mockito.any()))
    .thenAnswer(inv -> {
TestCase tc = inv.getArgument(0);
tc.setSuite(suite);
return tc;
});

// Act
TestCaseResponse response = testCaseService.add(suiteId, request);

// Assert
Assert.assertNotNull(response);
Assert.assertEquals(response.getTitle(), "Login test");
Assert.assertEquals(response.getRiskLevel(), "LOW");
Mockito.verify(testCaseRepository, Mockito.times(1)).save(Mockito.any());
Mockito.verify(llmService, Mockito.times(1))
    .generateTestCode(Mockito.any(), Mockito.any(), Mockito.any(), Mockito.any());
}
```